

LAPORAN TUGAS 5
SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyono M.Kom

Disusun Oleh:
Nama : Putri Suci Rahmadhani
Nim : 2410651008
Kelas : A
Tanggal : 8 November 2025

PRODI TEKNIK INFORMATIKA FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

1. SOAL PEMROGRAMAN DEADLOCK

- 1.1.k Kode Pemrograman
- 1.2. Analisis Masalah
- 1.3. Implementasi
- 1.4. Testing Output
- 1.5. Analisis Hasil

2. SOAL PEMROGRAMAN SOLUTION_RESOURCE

- 2.1.K Kode Pemrograman
- 2.2. Analisis Masalah
- 2.3. Implementasi
- 2.4. Testing Output
- 2.5. Analisis Hasil

3. SOAL PEMROGRAMAN

- 3.1. Kode Pemrograman
- 3.2. Analisis Masalah
- 3.3. Implementasi
- 3.4. Testing Output
- 3.5. Analisis Hasil

4. TABEL PERBANDINGAN EFISIENSI DAN FAIRNESS

- 4.1. Tabel Perbandingan Efisiensi
- 4.2. Tabel Perbandingan Fairness

5. ANALISIS MENDALAM

- 5.1. Mengapa Deadlock bisa terjadi?
- 5.2. Bagaimana masing-masing solusi bekerja?
- 5.3. Solusi mana yang paling adil?
- 5.4. Apakah ada kemungkinan starvation?

6. KESIMPULAN

1. SOAL PEMROGRAMAN DEADLOCK

1.1. Kode Pemrograman

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5

pthread_mutex_t chopstick[N];

void* philosopher(void* arg) {
    int id = (int)arg;

    while (1) {
        printf("Filsuf-%d sedang berpikir...\n", id);
        sleep(1);

        printf("Filsuf-%d lapar, mencoba ambil sumpit kiri (%d)...\n", id, id);
        pthread_mutex_lock(&chopstick[id]); // ambil kiri dulu

        printf("Filsuf-%d mencoba ambil sumpit kanan (%d)...\n", id, (id+1)%N);
        pthread_mutex_lock(&chopstick[(id+1)%N]); // ambil kanan

        printf("Filsuf-%d sedang MAKAN...\n", id);
        sleep(1);

        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id+1)%N]);

        printf("Filsuf-%d selesai makan, kembali berpikir...\n", id);
    }
}

int main() {
    pthread_t tid[N];
    int id[N];

    for (int i = 0; i < N; i++) pthread_mutex_init(&chopstick[i], NULL);
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &id[i]);
    }
    for (int i = 0; i < N; i++) pthread_join(tid[i], NULL);
}
```

1.2. Analisis Masalah

Pada program `dph_deadlock`, setiap filsuf mencoba mengambil dua sumpit untuk makan: sumpit kiri terlebih dahulu lalu sumpit kanan. Masalah muncul karena semua filsuf mengambil sumpit kiri pada waktu yang hampir bersamaan. Setelah itu, mereka semua menunggu sumpit kanan yang sedang dipegang oleh filsuf di sebelahnya. Akibatnya, tidak ada filsuf yang bisa mendapatkan pasangan sumpit lengkap, sehingga tidak ada yang bisa mulai makan. Setiap filsuf menunggu sumpit kanan yang tidak akan pernah dilepas karena filsuf lainnya juga sedang menunggu. Inilah yang menyebabkan deadlock, yaitu kondisi ketika semua proses saling menunggu dan tidak ada yang bisa melanjutkan. Masalah ini terjadi karena urutan pengambilan resource (sumpit) tidak diatur dengan baik, sehingga semua proses bisa terkunci bersamaan. Program ini menunjukkan bahwa tanpa mekanisme pencegah deadlock, sistem yang memakai banyak resource bersama dapat berhenti total.

1.3. Implementasi

Ini Adalah codingan final kenapa? Karena codingan diatas itu masih ada warning bisa dijalankan tapi lebih baik diperbaiki

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5

pthread_mutex_t chopstick[N];

void* filsuf(void* arg) {
    int id = *(int*)arg;

    while (1) {
        printf("Filsuf-%d sedang berpikir...\n", id);
        sleep(1);

        printf("Filsuf-%d lapar, mencoba ambil sumpit kiri (%d)...\n", id, id);
        pthread_mutex_lock(&chopstick[id]); // ambil kiri dulu

        printf("Filsuf-%d mencoba ambil sumpit kanan (%d)...\n", id, (id+1)%N);
        pthread_mutex_lock(&chopstick[(id+1)%N]); // ambil kanan

        printf("Filsuf-%d sedang MAKAN...\n", id);
        sleep(1);

        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id+1)%N]);

        printf("Filsuf-%d selesai makan, kembali berpikir...\n", id);
    }
}

int main() {
    pthread_t tid[N];
    int id[N];

    for (int i = 0; i < N; i++) pthread_mutex_init(&chopstick[i], NULL);
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, filsuf, &id[i]);
    }
    for (int i = 0; i < N; i++) pthread_join(tid[i], NULL);
}

```

1.4. Testing Output

```

madha_27@DESKTOP-558N811:~/praktikum-sinkronisasi/perobaan1$ nano dph_deadlock.c
madha_27@DESKTOP-558N811:~/praktikum-sinkronisasi/perobaan1$ gcc dph_deadlock.c -o dph_deadlock -pthread
madha_27@DESKTOP-558N811:~/praktikum-sinkronisasi/perobaan1$ ./dph_deadlock
Filsuf-0 sedang berpikir...
Filsuf-1 sedang berpikir...
Filsuf-2 sedang berpikir...
Filsuf-3 sedang berpikir...
Filsuf-4 sedang berpikir...
Filsuf-2 lapar, mencoba ambil sumpit kiri (2)...
Filsuf-2 mencoba ambil sumpit kanan (3)...
Filsuf-2 sedang MAKAN...
Filsuf-0 lapar, mencoba ambil sumpit kiri (0)...
Filsuf-0 mencoba ambil sumpit kanan (1)...
Filsuf-0 sedang MAKAN...
Filsuf-1 lapar, mencoba ambil sumpit kiri (1)...
Filsuf-3 lapar, mencoba ambil sumpit kiri (3)...
Filsuf-4 lapar, mencoba ambil sumpit kiri (4)...
Filsuf-4 mencoba ambil sumpit kanan (0)...
Filsuf-2 selesai makan, kembali berpikir...
Filsuf-2 sedang berpikir...
Filsuf-3 mencoba ambil sumpit kanan (4)...
Filsuf-4 sedang MAKAN...
Filsuf-0 selesai makan, kembali berpikir...
Filsuf-0 sedang berpikir...
Filsuf-1 mencoba ambil sumpit kanan (2)...
Filsuf-1 sedang MAKAN...
Filsuf-2 lapar, mencoba ambil sumpit kiri (2)...
Filsuf-4 selesai makan, kembali berpikir...
Filsuf-4 sedang berpikir...
Filsuf-3 sedang MAKAN...
Filsuf-0 lapar, mencoba ambil sumpit kiri (0)...
Filsuf-0 mencoba ambil sumpit kanan (1)...
Filsuf-0 sedang MAKAN...
Filsuf-1 selesai makan, kembali berpikir...
Filsuf-1 sedang berpikir...
Filsuf-2 mencoba ambil sumpit kanan (3)...
Filsuf-4 lapar, mencoba ambil sumpit kiri (4)...
Filsuf-4 mencoba ambil sumpit kanan (0)...
Filsuf-2 sedang MAKAN...
Filsuf-3 selesai makan, kembali berpikir...
Filsuf-2 sedang berpikir...
Filsuf-0 selesai makan, kembali berpikir...

```

1.5. Analisis Hasil

Semua sumpit mengambil kiri jadi yang terjadi Adalah semua silfut sling menunggu sumpit kanan karena sumpit kanan dipegang sebelahnya tidak ada yang bisa makan atau program terus berjalan hingga diberhentikan yaitu ctrl+c atau tidak ada baris mulai makan

2. SOAL PEMROGRAMAN SOLUTION_RESOURCE

2.1. Kode Pemrograman

```

#include <pthread.h>
#include <unistd.h>
#define N 5

pthread_mutex_t chopstick[N];
int makanCount[N] = {0};

void* philosopher(void* arg) {
    int id = (int)arg;
    int left = id;
    int right = (id + 1) % N;
    while (1) {
        printf("Filsuf-%d berpikir...\n", id);
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&chopstick[right]);
            pthread_mutex_lock(&chopstick[left]);
        } else {
            pthread_mutex_lock(&chopstick[left]);
            pthread_mutex_lock(&chopstick[right]);
        }

        printf("Filsuf-%d makan...\n", id);
        makanCount[id]++;
        sleep(1);
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);
    }
}

int main() {
    pthread_t tid[N];
    int id[N];
    for (int i = 0; i < N; i++) pthread_mutex_init(&chopstick[i], NULL);
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &id[i]);
    }

    sleep(60);

    sleep(60);
    printf("\n=== STATISTIK 60 DETIK ===\n");
    for (int i = 0; i < N; i++)
        printf("Filsuf-%d makan %d kali\n", i, makanCount[i]);
    printf("Tidak terjadi deadlock!\n");
}

```

2.2. Analisis Masalah

Pada program solusi Dining Philosophers, masalah utama yang ingin diatasi adalah terjadinya deadlock ketika semua filsuf mengambil sumpit dengan urutan yang sama. Jika setiap filsuf mengambil sumpit kiri terlebih dahulu lalu menunggu sumpit kanan, mereka bisa saling menahan resource dan tidak ada yang bisa makan. Untuk menghindari masalah tersebut, program solusi menggunakan pendekatan resource ordering, yaitu memberikan aturan urutan pengambilan sumpit yang berbeda pada satu filsuf. Dalam implementasi ini, empat filsuf tetap mengambil sumpit kiri dulu, namun satu filsuf terakhir mengambil sumpit kanan terlebih dahulu. Dengan perubahan sederhana ini, alur saling menunggu yang menyebabkan deadlock menjadi terputus. Hasilnya, tidak ada lagi kondisi di mana semua filsuf saling menahan resource sehingga program dapat berjalan lancar dan setiap filsuf bisa makan secara bergiliran. Analisis ini menunjukkan bahwa pengaturan urutan pengambilan resource secara konsisten adalah cara yang efektif untuk mencegah deadlock dalam sistem yang memakai resource bersama.

2.3. Implementasi

Ini Adalah codingan final karena codingan awal masih ada warningnya

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5

pthread_mutex_t chopstick[N];
int makanCount[N] = {0};

void* philosopher(void* arg) {
    int id = *(int*)arg;
    int left = id;
    int right = (id + 1) % N;
    while (1) {
        printf("Filsuf-%d berpikir...\n", id);
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&chopstick[right]);
            pthread_mutex_lock(&chopstick[left]);
        } else {
            pthread_mutex_lock(&chopstick[left]);
            pthread_mutex_lock(&chopstick[right]);
        }

        printf("Filsuf-%d makan...\n", id);
        makanCount[id]++;
        sleep(1);
        pthread_mutex_unlock(&chopstick[left]);
        pthread_mutex_unlock(&chopstick[right]);
    }
}

int main() {
    pthread_t tid[N];
    int id[N];
    for (int i = 0; i < N; i++) pthread_mutex_init(&chopstick[i], NULL);
    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &id[i]);
    }
}
```

2.4. Testing Output

```
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ nano solution_resource.c
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ gcc solution_resource.c -o solution_resource -pthread
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ ./solution_resource
Filsuf-0 berpikir...
Filsuf-1 berpikir...
Filsuf-2 berpikir...
Filsuf-3 berpikir...
Filsuf-4 berpikir...
Filsuf-0 makan...
Filsuf-2 makan...
Filsuf-2 berpikir...
Filsuf-1 makan...
Filsuf-0 berpikir...
Filsuf-3 makan...
Filsuf-1 berpikir...
Filsuf-3 berpikir...
Filsuf-2 makan...
Filsuf-4 makan...
Filsuf-2 berpikir...
Filsuf-4 berpikir...
Filsuf-3 makan...
Filsuf-0 makan...
Filsuf-3 berpikir...
Filsuf-2 makan...
Filsuf-0 berpikir...
Filsuf-4 makan...
Filsuf-2 berpikir...
Filsuf-1 makan...
Filsuf-4 berpikir...
Filsuf-3 makan...
Filsuf-1 berpikir...
Filsuf-2 makan...
Filsuf-3 berpikir...
Filsuf-4 makan...
^C
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$
```

2.5. Analisis Hasil

Setelah program dijalankan terlihat bahwa tidak ada filsuf yang berhenti aatau menunggu selamanya seperti pada deadlock.semua filsuf dapat berfikir mengambil sumpit dan makan lalu meletakan sumpit Kembali hal ini karena mengambil sumpit dengan urutan

berbeda sehingga alur saling menunggu dan menyebabkan deadlock berhasil diputus.

3. SOAL PEMROGRAMAN SOLUTION_LIMITING

3.1. Kode Pemrograman

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

pthread_mutex_t chopstick[N];
sem_t limit;
int makanCount[N] = {0};

void* philosopher(void* arg) {
    int id = *(int*)arg;

    while (1) {
        printf("Filsuf-%d berpikir...\n", id);
        sleep(1);

        sem_wait(&limit);

        pthread_mutex_lock(&chopstick[id]);
        pthread_mutex_lock(&chopstick[(id+1)%N]);

        printf("Filsuf-%d makan...\n", id);
        makanCount[id]++;
        sleep(1);

        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id+1)%N]);

        sem_post(&limit);
    }
}

int main() {
    pthread_t tid[N];
    int id[N];

    for (int i = 0; i < N; i++) pthread_mutex_init(&chopstick[i], NULL);

    for (int i = 0; i < N; i++) pthread_mutex_init(&chopstick[i], NULL);
    sem_init(&limit, 0, 4);

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &id[i]);
    }

    sleep(60);

    printf("\n=== STATISTIK 60 DETIK ===\n");
    for (int i = 0; i < N; i++)
        printf("Filsuf-%d makan %d kali\n", i, makanCount[i]);

    printf("Tidak terjadi deadlock!\n");
}
```

3.2. Analisis Masalah

Pada kasus Dining Philosophers, masalah yang muncul adalah semua filsuf bisa mencoba mengambil dua sumpit pada waktu yang bersamaan. Kondisi ini berpotensi menyebabkan deadlock, yaitu keadaan di mana semua filsuf memegang satu sumpit dan menunggu sumpit lainnya sehingga tidak ada yang bisa makan. Selain itu, jika tidak diatur, bisa terjadi starvation, yaitu kondisi ketika satu filsuf terus menunggu karena filsuf lain selalu lebih dulu mendapatkan sumpit. Masalah utama di versi ini adalah bagaimana membatasi

jumlah filsuf yang boleh mencoba makan pada saat yang sama agar situasi saling menunggu tidak terjadi.

3.3. Implementasi

Implementasi solution limiting biasanya dilakukan dengan membatasi jumlah filsuf yang boleh mengambil sumpit secara bersamaan, misalnya dengan menggunakan sebuah semaphore bernama room yang hanya mengizinkan empat filsuf masuk “ruang makan” pada satu waktu. Dengan begitu, tidak semua filsuf akan mengambil sumpit secara bersamaan. Setelah seorang filsuf selesai makan, ia keluar dari ruang makan dan “slot” untuk filsuf berikutnya terbuka kembali. Pendekatan ini memastikan alur makan lebih teratur, tidak ada situasi saling menunggu, dan setiap filsuf mendapat giliran.

3.4. Testing Output

```
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ nano solution_limiting.c
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ gcc solution_limiting.c -o solution_limiting -pthread
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ ./solution_limiting
Filsuf-0 berpikir...
Filsuf-1 berpikir...
Filsuf-2 berpikir...
Filsuf-3 berpikir...
Filsuf-4 berpikir...
Filsuf-0 makan...
Filsuf-2 makan...
Filsuf-0 berpikir...
Filsuf-4 makan...
Filsuf-1 makan...
Filsuf-2 berpikir...
Filsuf-1 berpikir...
Filsuf-3 makan...
Filsuf-0 makan...
Filsuf-4 berpikir...
Filsuf-3 berpikir...
Filsuf-2 makan...
Filsuf-0 berpikir...
Filsuf-4 makan...
Filsuf-2 berpikir...
Filsuf-3 makan...
Filsuf-4 berpikir...
Filsuf-1 makan...
Filsuf-3 berpikir...
Filsuf-0 makan...
Filsuf-2 makan...
Filsuf-1 berpikir...
Filsuf-2 berpikir...
Filsuf-0 berpikir...
Filsuf-4 makan...
Filsuf-1 makan...
Filsuf-3 makan...
Filsuf-4 berpikir...
Filsuf-0 makan...
Filsuf-1 berpikir...
Filsuf-3 berpikir...
Filsuf-2 makan...
Filsuf-4 makan...
Filsuf-0 berpikir...
^C
madha_27@DESKTOP-558N813:~/praktikum-sinkronisasi/percobaan1$ |
```

3.5. Analisis Hasil

Berdasarkan hasil keluaran program, semua filsuf dapat berpikir dan makan secara bergantian tanpa berhenti mendadak atau menunggu terlalu lama. Output menunjukkan bahwa filsuf yang makan bergantian secara acak, sesuai dengan cara kerja thread. Tidak ada filsuf yang terjebak menunggu sumpit selamanya, yang berarti deadlock berhasil dicegah. Selain itu, tidak terlihat ada filsuf yang terus-menerus berpikir tanpa mendapat giliran makan, sehingga starvation juga tidak terjadi. Hasil ini membuktikan bahwa pembatasan jumlah filsuf yang boleh mencoba makan sekaligus bekerja dengan baik dan membuat seluruh proses berjalan stabil, teratur, dan aman.

4. TABEL PERBANDINGAN EFISIENSI DAN FAIRNES

4.1. Tabel perbandingan efisiensi

Aspek	Versi Deadlock	Resources ordering	Limiting philosophers
Kemungkinan deadlock	Sangat tinggi	Tidak ada	Tidak ada
Throughput	0 (macet)	Stabil	Stabil
Konsumsi CPU	Rendah (karena macet)	Normal	Normal
Stabilitas program	Tidak stabil	Stabil	Sangat stabil
Kemampuan skalabilitas	Buruk	Baik	Baik
Efisiensi waktu eksekusi	Tidak efisien	Efisien	Efisien

4.2. Tabel perbandingan fairness

Aspek	Versi deadlock	Resource ordering	Limiting philosophers
Semua filsuf bisa makan	Tidak	Bisa	Bisa
Distribusi kesempatan	Tidak ada	Cukup merata	Lebih merata
Risiko starvation	Tinggi	Rendah	Sangat rendah
Pemerataan jumlah makan	Tidak ada	Baik	Sangat baik
Keadilan akses sumber daya	Gagal	Baik	Sangat baik

5. ANALISIS MENDALAM

5.1. Mengapa Deadlock bisa terjadi?

Karena semua proses (filsuf) saling menunggu sumber daya (sumpit) yang sedang dipegang oleh proses lain.

Pada versi deadlock:

Setiap filsuf mengambil sumpit kiri terlebih dahulu.

Setelah memegang satu sumpit, mereka menunggu sumpit kanan.

Tetapi sumpit kanan sudah diambil oleh filsuf berikutnya.

Akhirnya, semua filsuf memegang 1 sumpit dan menunggu sumpit lain yang tidak pernah dilepas.

Dalam teori deadlock, keempat syarat berikut terpenuhi:

1. Mutual Exclusion → sumpit hanya bisa dipakai satu filsuf.
 2. Hold and Wait → filsuf memegang satu sumpit sambil menunggu sumpit lain.
 3. No Preemption → sumpit tidak bisa diambil paksa.
 4. Circular Wait → terjadi lingkaran saling menunggu ($F_0 \rightarrow F_1 \rightarrow F_2 \rightarrow F_3 \rightarrow F_4 \rightarrow F_0$).
- Karena keempat kondisi ini aktif, deadlock DIJAMIN terjadi.

5.2. Bagaimana masing-masing solusi bekerja?

Solusi 1: Resource Ordering

Ide: Jangan biarkan semua filsuf mengambil sumpit dengan urutan yang sama.

Filsuf genap: ambil sumpit kanan → lalu kiri

Filsuf ganjil: ambil sumpit kiri → lalu kanan

Ini membuat urutan pengambilan sumpit tidak simetris, sehingga circular wait terputus.

Hasil:

- Tidak ada deadlock
- Kinerja stabil
- Pembagian makan cukup merata

Solusi 2: Limiting Philosophers (Semaphore)

Ide: Batasi jumlah filsuf yang boleh mencoba makan secara bersamaan.

Hanya 4 filsuf yang boleh mengambil sumpit pada waktu yang sama (`sem_wait`).

Selalu ada minimal 1 filsuf yang tidak memegang sumpit, sehingga lingkaran menunggu tidak pernah lengkap.

Hasil:

- Tidak ada deadlock
- Tidak ada circular wait
- Distribusi makan menjadi sangat merata
- Risiko starvation sangat kecil

5.3. Solusi mana yang paling adil?

Berdasarkan hasil percobaan:

Solusi paling adil: Limiting Philosophers

Karena semua filsuf mendapatkan kesempatan makan yang hampir sama tidak ada filsuf yang terlalu lama menunggu penggunaan semaphore menjamin gilirannya seimbang sedangkan Resource Ordering tetap memungkinkan satu filsuf makan sedikit lebih sering, tapi masih cukup adil.

5.4. Apakah ada kemungkinan starvation?

- Versi Deadlock:

Starvation pasti terjadi (bahkan semua filsuf tidak bisa makan sama sekali).

- Resource Ordering:

Starvation kecil kemungkinan, tetapi masih mungkin. misalnya jika jadwal thread tidak seimbang walau kecil, kemungkinan starvation tetap ada.

- Limiting Philosophers:

Starvation hampir tidak mungkin karena semua filsuf dibatasi bergiliran oleh semaphore. sistem memastikan maksimal 4 filsuf mencoba makan, sehingga antrian tetap bergerak Filsuf tidak bisa dikunci terlalu lama. Ini solusi paling aman dari starvation.

6. KESIMPULAN

Deadlock terjadi karena beberapa proses saling menunggu resource yang dipegang proses lain, sehingga semuanya berhenti dan tidak ada yang bisa jalan. Setiap solusi yang dipakai intinya berusaha mengatur giliran supaya proses tidak berebut resource. Ada yang pakai mutex, ada yang pakai aturan urutan pengambilan resource, dan ada juga yang pakai sistem “nunggu dulu” (waiter).

Dari semua cara, solusi yang mengatur urutan pengambilan resource atau pakai waiter biasanya paling adil karena semua proses dapat giliran secara merata. Walaupun begitu, tetap saja ada kemungkinan starvation jika ada proses yang terus-terusan kalah giliran atau tidak pernah dijadwalkan.